DOCUMENTAÇÃO DE TESTES

Módulo de cadastro de voluntários — VolunJá API

Arquivo de referência	tests/auth/test_voluntarios.py
Frameworks e bibliotecas	pytest, unittest.mock, FastAPI TestClient, PyJWT e Argon2
Escopo documentado	Serviço, repositório, endpoint HTTP e dependências do módulo
Quantidade de cenários	26 casos considerando as variações parametrizadas

1. Objetivo

Esta documentação descreve os testes automatizados apresentados no arquivo tests/auth/test_voluntarios.py. O conjunto verifica o fluxo de cadastro de voluntários, as regras de negócio, a persistência por meio do repositório, a resposta HTTP da API e a configuração de dependências do módulo.

O documento registra o comportamento que os testes procuram garantir. Ele não representa, por si só, um relatório de execução: o arquivo fornecido contém o código dos testes, mas não apresenta o resultado de uma execução do pytest.

2. Tecnologias e recursos utilizados

Recurso	Finalidade nos testes
pytest	Organização, execução, fixtures e parametrização dos cenários.
unittest.mock.MagicMock	Simulação da conexão com o banco e do cursor SQL.
FastAPI TestClient	Envio de requisições HTTP ao endpoint de cadastro.
PyJWT (jwt)	Decodificação e inspeção das informações do token JWT emitido.
argon2.PasswordHasher	Verificação de que a senha é armazenada como hash Argon2.

3. Preparação dos dados e simulações

Os testes utilizam dados fixos para tornar os cenários reproduzíveis. O corpo padrão da requisição representa Maria de Souza, com e-mail maria@teste.com, identificador 99988877766, senha de teste e consentimento dos termos habilitado.

O registro simulado do banco contém os campos id, nome_completo, email, identificacao, password_hash, consentimento_termos, perfil e ativo. O repositório em memória mantém um voluntário previamente cadastrado para permitir a verificação de duplicidade.

Fixtures: repo_mock fornece o repositório em memória e service instancia o serviço utilizando esse repositório. As funções auxiliares _requisicao, _client e _conn_com_resultado simplificam a criação de requisições, clientes HTTP e conexões simuladas.

4. Casos de teste

4.1. Camada de serviço

test_cadastro_retorna_dto_publico_com_perfil_fixo

Verifica se o cadastro retorna um DTO público com nome e e-mail esperados, perfil fixo “Voluntario”, tipo de token Bearer e validade de 60 minutos. Também confere se o registro foi salvo como ativo e com consentimento.

test_cadastro_emite_jwt_de_60_minutos_sem_dados_sensiveis

Decodifica o JWT e verifica subject, e-mail, perfil e intervalo de validade de uma hora. Confere que os campos password e password_hash não aparecem no payload.

test_cadastro_normaliza_email_para_minusculas

Envia um e-mail com letras maiúsculas e verifica se o cadastro pode ser localizado pela versão em letras minúsculas.

test_cadastro_rejeita_e_nao_persiste

Teste parametrizado para três situações: e-mail já cadastrado, identificador já cadastrado e ausência de consentimento. Cada situação deve lançar sua exceção de negócio correspondente e não aumentar a quantidade de registros.

test_senha_e_armazenada_como_hash_argon2

Verifica que a senha salva é diferente da senha original, possui prefixo Argon2 e pode ser validada com a senha correta. A senha incorreta deve produzir VerifyMismatchError.

4.2. Camada de repositório

test_repositorio_buscar_por_email_encontrado

Simula um registro retornado pelo banco. Verifica a criação de VoluntarioEntity, a conversão dos campos booleanos e o envio do e-mail em letras minúsculas ao comando SQL.

test_repositorio_converte_flags_para_bool

Confere a conversão dos valores 0 de consentimento_termos e ativo para False.

test_repositorio_buscar_por_email_nao_encontrado

Verifica que a busca por um e-mail inexistente retorna None.

test_repositorio_buscar_por_identificador_encontrado

Verifica a busca por identificador e a remoção de espaços em branco antes de enviar o parâmetro à consulta SQL.

test_repositorio_buscar_por_identificador_nao_encontrado

Verifica que a busca por identificador sem correspondência retorna None.

test_repositorio_salvar_executa_insert_e_commit

Confere que salvar retorna a mesma entidade, normaliza o e-mail no INSERT e chama commit uma vez.

test_repositorio_sem_conexao_nao_acessa_banco

Verifica o comportamento quando o repositório é criado sem conexão: as buscas retornam None e salvar retorna a entidade recebida.

4.3. Camada HTTP / router

test_router_cadastro_retorna_201_sem_expor_senha

Envia POST para /api/v1/voluntarios. Verifica HTTP 201, perfil do voluntário, token retornado e ausência da senha no corpo da resposta e no objeto público.

test_router_converte_excecoes_de_negocio

Teste parametrizado: e-mail duplicado e identificador duplicado devem resultar em HTTP 409; falta de consentimento deve resultar em HTTP 400. O detalhe da resposta deve corresponder à mensagem da exceção.

test_router_rejeita_entrada_invalida_com_422

Teste parametrizado para senha curta, e-mail inválido, nome curto e identificador curto. Cada entrada inválida deve resultar em HTTP 422.

test_router_rejeita_corpo_sem_consentimento_com_422

Remove o campo consentimento_termos do corpo enviado e verifica que a validação da requisição retorna HTTP 422.

test_router_get_db_abre_e_fecha_conexao

Substitui a função de conexão por um mock, verifica que get_db disponibiliza a conexão e que o fechamento do gerador fecha a conexão uma vez.

test_router_get_voluntario_service_monta_service

Verifica se get_voluntario_service, recebendo uma conexão simulada, retorna uma instância de VoluntarioService.

4.4. Integração de configuração da aplicação

test_main_registra_rota_de_voluntarios — importa a aplicação principal, verifica que o objeto é uma instância de FastAPI e consulta o OpenAPI para confirmar o registro do caminho /api/v1/voluntarios com o método POST.

5. Matriz resumida de cobertura

Área	O que é verificado	Casos parametrizados
Serviço	Cadastro, DTO, JWT, normalização, duplicidade, consentimento e hash de senha.	3 variações de rejeição
Repositório	Consultas por e-mail/identificador, conversões, INSERT, commit e ausência de conexão.	Nenhum
Router	Status HTTP, resposta pública, exceções, validação e ciclo de vida da conexão.	3 grupos: 3 + 4 variações
Aplicação	Registro da rota POST no OpenAPI.	Nenhum

6. Como executar os testes

Na raiz do projeto, execute o arquivo de testes com:

```
pytest tests/auth/test_voluntarios.py -v
```

Para executar toda a suíte de testes do projeto:

```
pytest -v
```

7. Critérios de aprovação observados no código

• O cadastro válido deve produzir uma resposta com perfil “Voluntario” e token Bearer.

• O JWT deve ter validade de 60 minutos e não deve conter senha ou hash de senha.

• A senha deve ser armazenada como hash Argon2, verificável com a senha correta.

• E-mail e identificador duplicados devem ser rejeitados sem persistência adicional.

• O consentimento deve ser obrigatório; sua ausência é tratada pela regra de negócio ou pela validação do corpo, conforme o cenário.

• O repositório deve normalizar e-mail e identificador conforme os cenários testados.

• O endpoint deve retornar os status HTTP definidos nos testes: 201, 409, 400 e 422.

• A rota POST /api/v1/voluntarios deve estar registrada na aplicação.

8. Limites da documentação

A documentação foi elaborada exclusivamente a partir do código de testes fornecido. O arquivo não demonstra testes de carga, testes de concorrência, integração com um banco MySQL real, testes de autorização de rotas protegidas ou cobertura de outros módulos. Também não foi fornecido um relatório de execução; portanto, não é possível afirmar que todos os testes passaram.

Fonte: arquivo de testes fornecido pelo usuário — tests/auth/test_voluntarios.py.